Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-26

Яшков П. А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы

Практическое освоение разработки eBPF-зондов на языке C, мониторинга системных вызовов Linux с помощью BCC, фильтрации событий в пространстве ядра и ведения журнала предупреждений на Python.

Индивидуальное задание — вариант №10

По правилу методички номер варианта вычисляется как (24 mod 15) + 1 = 10. Требуется обнаруживать любую попытку системного вызова init_module, предназначенного для загрузки модулей ядра Linux, посредством tracepoint syscalls:sys_enter_init_module. Для задания 1 общий системный вызов — execve.

Показатель	**Содержание**
Номер в журнале / вариант	24 / 10
Основной tracepoint	syscalls:sys_enter_init_module
Индивидуальное условие	Любая попытка вызова init_module, независимо от результата
Поля события	PID, UID, image_len, comm
Механизм передачи	BCC BPF_PERF_OUTPUT (perf ring buffer)
Место сохранения	logs/security_audit.log

Сведения о проверках

Реальный BCC на этой вычислительной среде не установлен, а tracepoint sys_enter_init_module отсутствует. Поэтому здесь проверены только C-логика с тестовыми заголовками, Python-монитор без BCC и попытка syscall с заведомо неправильным содержимым. Ни одна из приведённых иллюстраций не выдаётся за скриншот работающего eBPF-приложения. Для полной сдачи требуются живые демонстрации в Ubuntu/WSL2 с поддержкой BCC.

Задание 1. Базовый перехват запуска процессов

Разработан C-зонд, использующий TRACEPOINT_PROBE(syscalls, sys_enter_execve). При входе процесса в execve вычисляются PID/TGID и UID при помощи bpf_get_current_pid_tgid() и bpf_get_current_uid_gid(); имя выполняемой задачи получают bpf_get_current_comm(). Структура передаётся через execve_events.perf_submit().

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

/* Событие запуска процесса для передачи из ядра в Python. */
struct execve_event_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
};

BPF_PERF_OUTPUT(execve_events);

TRACEPOINT_PROBE(syscalls, sys_enter_execve)
{
    struct execve_event_t event = {};
    event.pid = (u32)(bpf_get_current_pid_tgid() >> 32);
    event.uid = (u32)(bpf_get_current_uid_gid() & 0xffffffffULL);
    bpf_get_current_comm(&event.comm, sizeof(event.comm));
    execve_events.perf_submit(args, &event, sizeof(event));
    return 0;
}
```

Пользовательский скрипт main.py загружает исходник BPF(text=...), связывает perf-буфер с monitor.handle_execve_event и опрашивает события. Для подтверждения требуются команды:

```
# Терминал A (Ubuntu):
sudo /usr/bin/python3 main.py --task 1
# Терминал B:
/bin/ls -l
/bin/cat /etc/os-release
```

На точке входа sys_enter_execve поле comm может ещё обозначать имя прежней задачи. Для точного пути запускаемого файла нужен аргумент filename либо событие sched_process_exec. Наблюдение execve в реальном ядре здесь пока не выполнено.

Задание 2. Мониторинг аномального поведения — init_module

Для варианта №10 разработан C-зонд, прикреплённый ТОЛЬКО к системному событию sys_enter_init_module. Это и есть фильтр в ядре по типу вызова: все остальные системные вызовы не запускают данный зонд и не отправляют события в пользовательское пространство. В отличие от вариантов с условием сравнения аргумента, здесь дополнительный оператор if для отбрасывания событий не нужен — по условию регистрируется ЛЮБАЯ попытка, включая image_len = 0.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

/* Вариант 10. Любая попытка вызова init_module(2) считается событием.
 * Хук привязан только к sys_enter_init_module: ядро не посылает сюда
 * события от остальных системных вызовов. Нельзя отбрасывать ошибки,
 * поскольку на входе результат вызова ещё неизвестен.
 */
struct init_module_event_t {
    u32 pid;                /* TGID процесса-инициатора */
    u32 uid;                /* UID */
    u64 image_len;          /* Длина переданного образа модуля */
    char comm[TASK_COMM_LEN];   /* Имя текущей задачи */
};

BPF_PERF_OUTPUT(module_events);

TRACEPOINT_PROBE(syscalls, sys_enter_init_module)
{
    /* Фильтрация В ЯДРЕ реализована выбором точного tracepoint:
     * сюда попадают ТОЛЬКО попытки init_module, а не все syscall.
     * Передавать событие следует при ЛЮБОМ значении len, включая 0.
     */
    struct init_module_event_t event = {};
    event.pid = (u32)(bpf_get_current_pid_tgid() >> 32);
    event.uid = (u32)(bpf_get_current_uid_gid() & 0xffffffffULL);
    event.image_len = (u64)args->len;
    bpf_get_current_comm(&event.comm, sizeof(event.comm));
    module_events.perf_submit(args, &event, sizeof(event));
    return 0;
}
```

Данные события включают PID/UID инициатора, длину переданного образа (image_len) и текущее имя comm. Копирование произвольного образа модуля в Python не выполняется, так как для поставленной задачи достаточно телеметрии. sys_enter фиксирует только вызов и не сообщает, разрешил ли Linux загрузку.

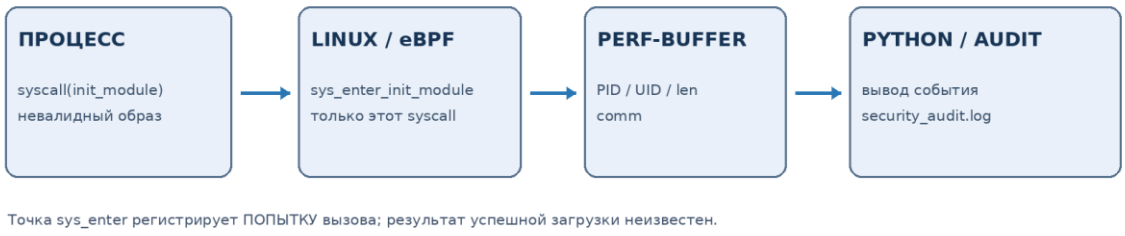


Рисунок 1 — Архитектура сбора и передачи событий варианта 10 (схема)

Проверка алгоритма вне ядра

Применён GCC host mock с подставными типами и функциями BCC. Для init_module проверены три входных значения len: 4, 0 и 1048576 байт; все три попытки приводят к одному событию с правильными полями. Дополнительно проверен execve-зонд. Это проверка C-алгоритма, а не успешная компиляция BPF или прохождение проверки в ядре.

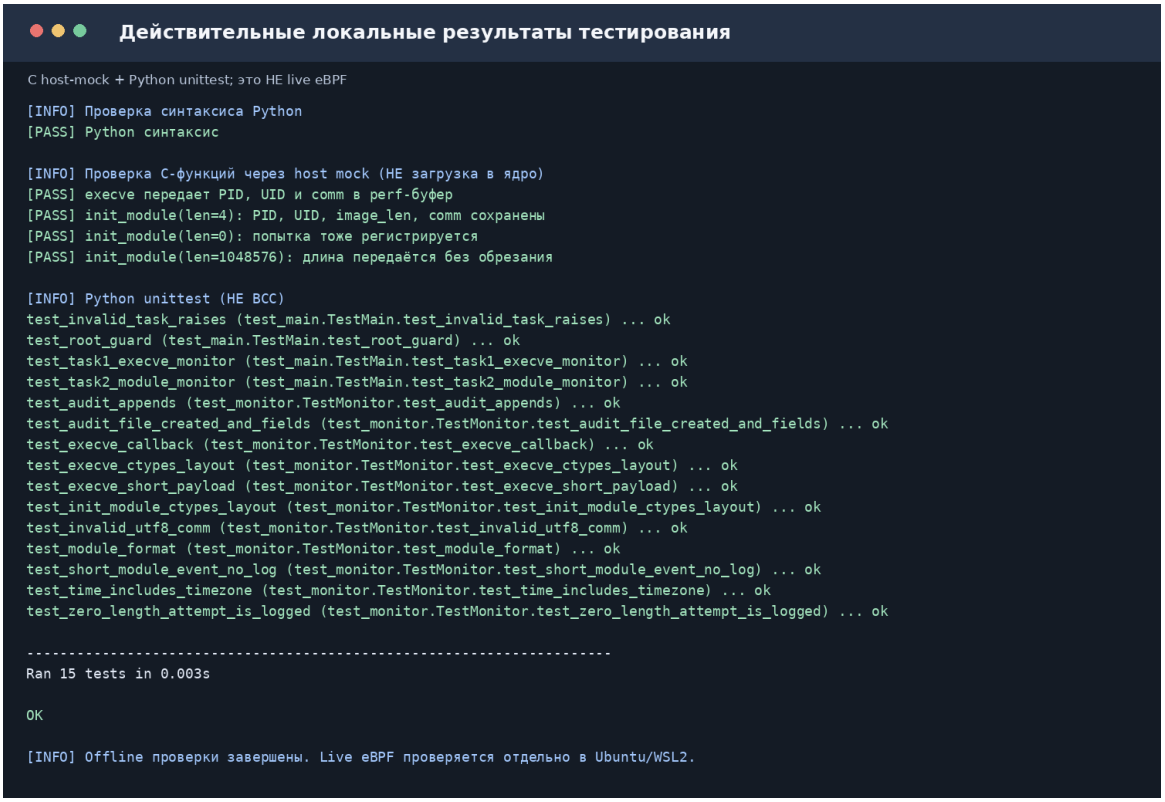


Рисунок 2 — Реальный вывод локальных C/Python тестов, БЕЗ загрузки eBPF

Безопасное воспроизведение попытки загрузки модуля

Включён генератор scripts/generate_safe_init_module_attempt.py. Он вызывает syscall init_module с содержимым NOT_A_MODULE — коротким недействительным буфером, который не является .ko-файлом. На доступной среде вызов был отклонён с ENOSYS; успешной загрузки модуля не было. Это подтверждает только попытку обращения к системному вызову и ответ ОС, но не факт прохождения tracepoint.

```
# Пока монитор задания 2 работает в терминале A,
# выполнить в терминале B:
/usr/bin/python3 scripts/generate_safe_init_module_attempt.py
```

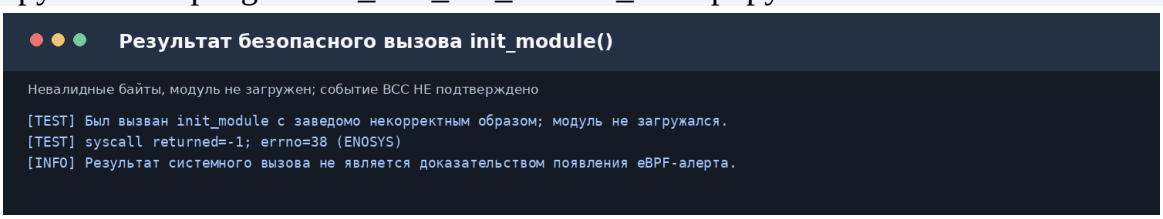


Рисунок 3 — Фактический локальный syscall-результат ENOSYS; без BCC

В Linux может сработать ограничение seccomp ещё до трассировки; в таком случае eBPF-алерт отсутствует даже при корректном коде. На ядрах без CONFIG_MODULES могут отсутствовать необходимый syscall и tracepoint. Для окончательной демонстрации нужен совместимый Linux-хост с BCC.

Задание 3. Python-монитор и журнал аудита

События module_events обрабатываются ctypes-структурой InitModuleEvent с полями того же размера и порядка, что в C. callback добавляет временную метку, собирает строку предупреждения и дописывает её в security_audit.log. В журнале сохраняются Timestamp, PID, UID, image_len и имя процесса.

```
class InitModuleEvent(ctypes.Structure):
    _fields_ = [
        ("pid", ctypes.c_uint32),
        ("uid", ctypes.c_uint32),
        ("image_len", ctypes.c_uint64),
        ("comm", ctypes.c_char * 16),
    ]

def handle_module_event(_cpu, data, size):
    if size < ctypes.sizeof(InitModuleEvent):
        return
    event = ctypes.cast(data, ctypes.POINTER(InitModuleEvent)).contents
    line = format_module_event(event, timestamp())
    print(line, flush=True)
    LOG_FILE.parent.mkdir(parents=True, exist_ok=True)
    with LOG_FILE.open("a", encoding="utf-8") as out:
        out.write(line + "\n")
```

Запись формата отдельно проверена через искусственно созданное ctypes-событие (PID=8702, UID=1000, image_len=13). Это синтетический тест Python callback; он НЕ является перехватом вызова реальным eBPF-монитором и не должен подаваться как таковой.

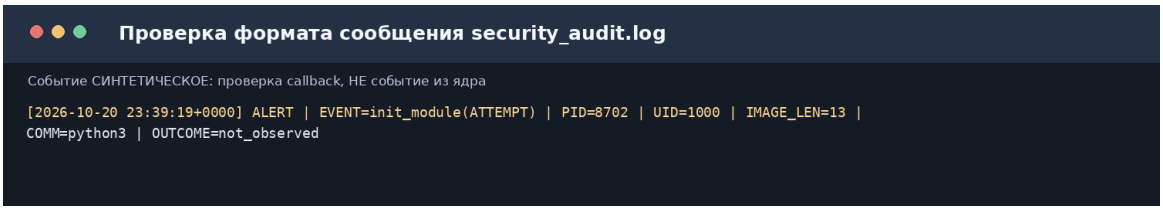


Рисунок 4 — СИНТЕТИЧЕСКИЙ тест сохранения security_audit.log; не live eBPF

Фактический live-журнал будет создан по адресу logs/security_audit.log после запуска: sudo /usr/bin/python3 main.py --task 2, если kernel tracepoint активен и тестовый syscall достиг его. Чтобы полностью выполнить требования методички, нужно показать сам монитор, вызов в соседнем терминале и содержимое этого файла.

Задание 4. Проектирование профилей защиты

4.1. Seccomp-BPF и eBPF (kprobes/tracepoints)

Критерий	Seccomp-BPF	eBPF tracepoint/kprobe
Предназначение	Принудительная фильтрация syscalls	Наблюдение, аудит, диагностика
Момент работы	Перед выполнением действия ядром	На контрольной точке вызова или в функции ядра
Последствие	Может запретить вызов или вернуть ошибку	Обычно формирует событие; не отменяет вызов
Для init_module	Правила могут запретить системный вызов по номеру	Можно зарегистрировать попытку init_module
Учет результата	Политика принимает решение до выполнения	sys_enter не знает возврат syscall

Seccomp-BPF предназначен для песочниц и контролирует выполнение syscall на границе перехода в ядро. Обработчик может запретить системный вызов, вернуть ошибку либо потребовать пользовательского решения. В отличие от него программа на sys_enter_init_module только записывает событие. Изменять управление системным вызовом из обычного kprobe/tracepoint не предусмотрено: произвольная модификация состояния ядра опасна и может нарушить согласованность данных. Для блокировки подходят Seccomp-BPF и политики LSM.

4.2. TOCTOU для init_module и методы противодействия

TOCTOU — разрыв между проверкой данных и их последующим использованием. У init_module аргумент umod — указатель на образ модуля в памяти процесса, len — скалярная длина, uargs — указатель на строку параметров. Другой поток того же процесса теоретически способен изменить память, на которую указывает umod/uargs, между чтением монитором и копированием ядром. Поэтому сравнение текста, прочитанного отдельно, не должно считаться непреодолимым контролем доступа.

Разумный способ снижения риска в eBPF — считывать пользовательскую строку ограниченного размера через bpf_probe_read_user_str(), проверять код возврата и работать с копией в локальном буфере; для бинарного образа использовать bpf_probe_read_user() с жёсткой верхней границей. Это защищает сам процесс анализа от повторного чтения

изменившегося указателя, но не делает снимок обязательным для дальнейшего исполнения ядром. Контроль разрешений должен выполняться в точке решения ядра (например, LSM), а не post-factum в мониторе.

В разработанном варианте целевой критерий — сам факт вызова init_module, поэтому образ по указателю не считывается. image_len сохраняется как полученный на входе числовой аргумент. Отдельное ограничение: этот зонд не отслеживает другой Linux syscall finit_module, который может использоваться некоторыми утилитами.

Сводка результатов и перечень недостающих live-снимков

Этап	**Фактически подтверждено**	**Что нужно для сдачи**
1. execve	C host mock, Python callback	Live-скриншот монитора после ls/cat
2. init_module	3 проверки C, syscall с ошибкой ENOSYS	Live-скриншот BCC-алерта и второго терминала
3. security_audit.log	Формат/запись проверены на synthetic payload	Live-файл, созданный реальным BCC-событием
4. Seccomp/TOCTOU	Теория подготовлена	Без дополнительных испытаний

Команды для окончательной проверки в Ubuntu

```
# Сначала убедиться в существовании tracepoint:
sudo ls /sys/kernel/tracing/events/syscalls/sys_enter_init_module

# Терминал A, задание 1:
sudo /usr/bin/python3 main.py --task 1
# Терминал B: /bin/ls -l ; /bin/cat /etc/os-release

# После Ctrl+C, терминал A, задания 2-3:
sudo /usr/bin/python3 main.py --task 2
# Терминал B:
/usr/bin/python3 scripts/generate_safe_init_module_attempt.py
# Терминал C:
sudo cat logs/security_audit.log
```

Если драйвер tracing или BCC отсутствует, следует использовать полноценную VM Ubuntu с подходящим ядром. Автономные изображения являются действительными результатами сохранённых проверок, но не заменяют три живых изображения, требуемые методичкой.

Вывод: Подготовлены C-зонды и Python-монитор по лабораторной №3, варианту 10 (execve, sys_enter_init_module), механизм отправки событий в perf-буфер и журнал security_audit.log. Успешно выполнены четыре C-проверки и пятнадцать Python-тестов. Зафиксирована попытка системного вызова с недействительным образом (ENOSYS). Работа eBPF в ядре и скриншоты реальных событий остаются не подтверждены в данной среде и должны быть получены в Linux с поддержкой BCC.